

RC-Fernsteuerung auf Arduino Mega + Display-Nano

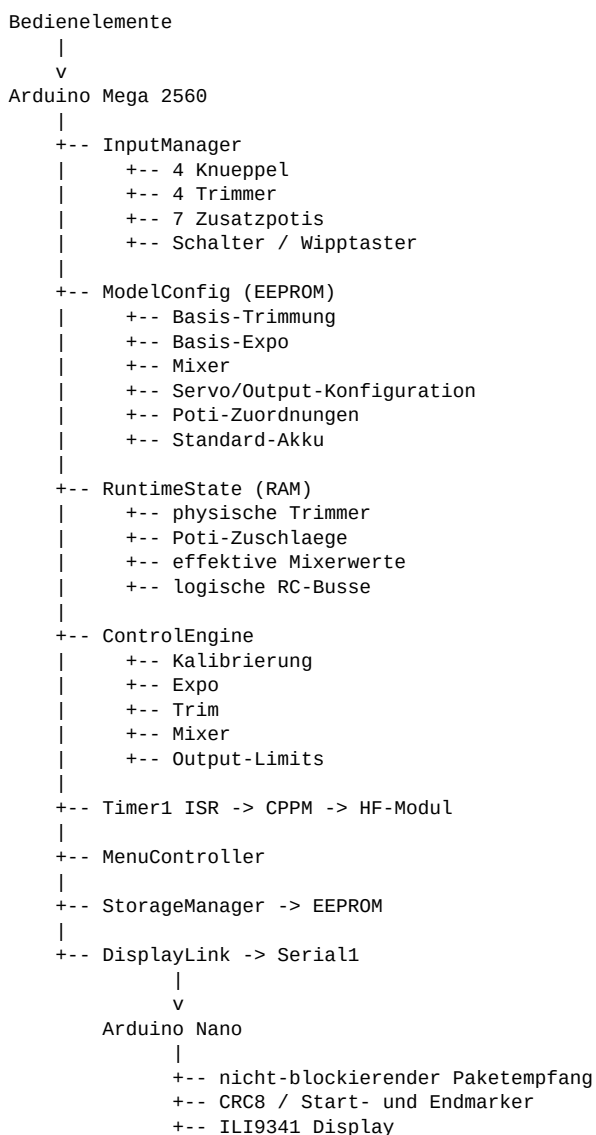
1. Ziel des Entwurfs

Der Entwurf fuehrt die beiden vorhandenen Softwarelinien zusammen:

- Die bestehende RC-Implementierung liefert die bereits erprobten Modellprofile, Ruderausschlaege, Expo-Werte, Mischfunktionen, Akku-Grunddaten und die grundsaeztliche PPM/CPPM-Idee.
- Das Mega-Projekt liefert die neue Pinbelegung mit A0 bis A14, Timer-1-CPPM auf Pin 3 sowie die bereits optimierte serielle Display-Kommunikation zum Nano.
- Die neue Architektur ergaenzt Modellverwaltung im EEPROM, eine hierarchische Menu-State-Machine und klar getrennte Runtime-Korrekturen fuer Trimmung, Expo, Mixer und Zusatzfunktionen.

Der Arduino Mega 2560 ist alleinige Quelle der RC-Wahrheit. Der Nano besitzt keine Modellkonfiguration; er zeigt nur den vom Mega gelieferten UI-Zustand an.

2. Systemarchitektur



+-+ partielles Redraw fuer Monitorbalken
+-+ Summer

3. Grundregel: Basiswert und Runtime-Wert

Viele Parameter werden waehrend des Betriebs veraendert. Deshalb werden gespeicherte Modellwerte und physische Regler strikt getrennt.

EEPROM-Basiswert + Runtime-Korrektur = effektiver Wert

Beispiel Expo:

Basis-Expo Quer	30 %
Poti-Korrektur	+12 %

Effektives Expo	42 %

Poti- und Trimmerbewegungen schreiben niemals automatisch in den EEPROM. Nur Aenderungen im Konfigurationsmenu markieren das aktive Modell als dirty. Nach 750 ms ohne weitere Konfigurationsaenderung wird das Modell gespeichert.

4. Analoge Eingänge

Knueppel

- A0: Gas
- A2: Seite
- A4: Hoehe
- A6: Quer

Die vorhandenen Kalibrier Grenzen aus dem Mega-Projekt werden als Defaultwerte uebernommen. Eine kleine Totzone um die Mittelstellung verhindert ADC-Jitter um Null.

Physische Trimmer

- A1: Gas-Trim
- A3: Seiten-Trim
- A5: Hoehen-Trim
- A7: Quer-Trim

Die Trimmer werden gelaettet und als Runtime-Offset auf die im Modell gespeicherte Basisstrimmung addiert. Der Standard-Regelbereich ist +/-25 Prozent und kann spaeter pro Achse parametrisiert werden.

Zusatzpotis

- A8 bis A14: sieben frei zuweisbare Runtime-Regler

A8 wird wegen des ADC-Multiplexerwechsels A7 -> A8 zweimal gelesen; die erste Konvertierung wird verworfen. Alle sieben Zusatzpotis verwenden einen einfachen IIR-Filter.

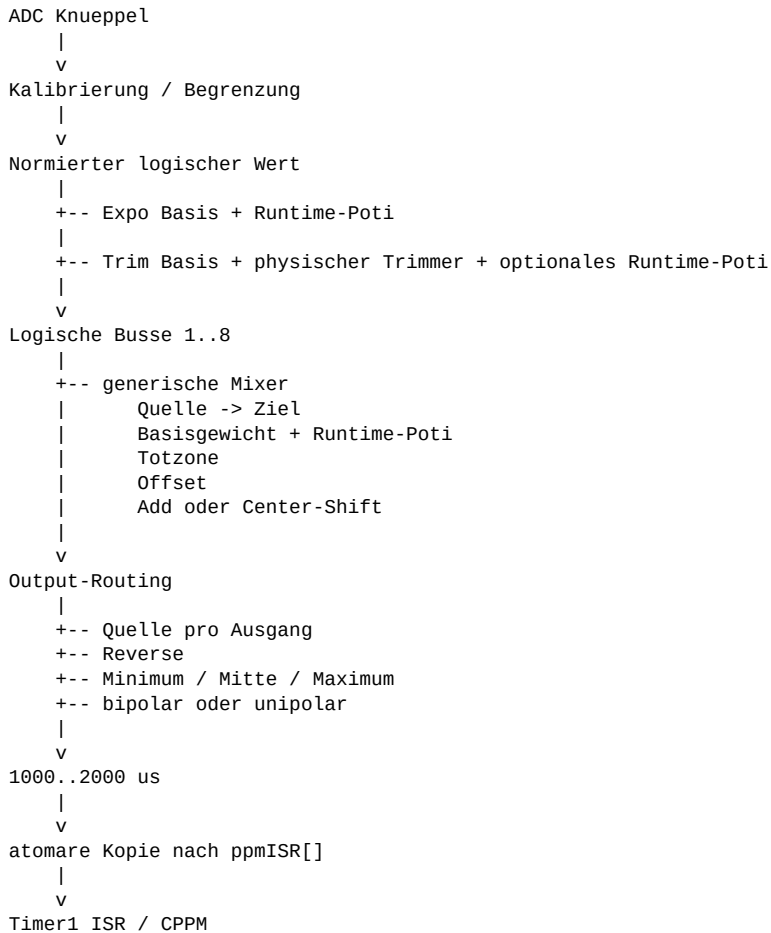
Eine Poti-Zuordnung besteht aus:

- Funktion

- Minimum-Offset
- Maximum-Offset
- Reverse

Moegliche Funktionen sind Expo, Trim, Mixer 1 bis 8 und Aux 1 bis 4.

5. RC-Signalpipeline



Die CPPM-Erzeugung bleibt damit von Menu und Display entkoppelt. Die ISR liest nur den bereits fertig berechneten Puffer.

6. Logische Busse und Ausgaenge

Es existieren acht logische Busse:

1. Gas
2. Seite
3. Hoehe
4. Quer
5. Aux1
6. Aux2
7. Aux3
8. Aux4

Jeder der acht CPPM-Ausgaenge kann einen beliebigen Bus als Quelle verwenden. Dadurch sind z. B. zwei getrennte Querruderservos oder zwei Flapservos moeglich, ohne die Trimmer als eigene CPPM-Kanaele zu verschwenden.

Die Defaultbelegung orientiert sich am alten System:

- CH1: Gas
- CH2: Hoehe
- CH3: Seite
- CH4: Quer links
- CH5: Quer rechts
- CH6: Flap links / Aux1
- CH7: Flap rechts / Aux1
- CH8: Aux2

7. Uebernommene Modellprofile

Beim erstmaligen Initialisieren des EEPROM werden vier bekannte Modelle angelegt:

1. Multiplex Heron
2. DPower 270X
3. Graupner EPS
4. Amigo 4

Uebernommen wurden insbesondere die alten Ruderausschlaege, Richtungsinformationen, Expo-Ausgangswerte und die wichtigsten Mischanteile. Die alten Expo-ADC-Werte werden fuer den neuen Entwurf in Prozentnaeherungen ueberfuehrt.

Weitere Modellplaetze sind leer und koennen durch Auswahl aus der Modellliste als Kopie des aktuell aktiven Modells angelegt werden.

8. Modellverwaltung und EEPROM

Der EEPROM speichert:

```
SystemConfig
  activeModelId
  activeBatteryId

ModelConfig[12]
BatteryConfig[16]
```

Alle Datensaeetze besitzen Magic, Versionsnummer und CRC16. Bei einem Versionswechsel wird die alte Struktur nicht stillschweigend als neue Konfiguration interpretiert.

Beim Einschalten:

```
EEPROM lesen
  |
  v
activeModelId pruefen
  |
  v
aktives Modell laden
  |
```

```

      v
Runtime aus physischen Reglern berechnen
      |
      v
Home-Screen

```

Ein anderes Modell wird erst nach expliziter Aktivierung geladen. Vor dem Wechsel wird eine ausstehende Konfigurationsaenderung des aktuellen Modells gespeichert. Anschliessend werden `activeModelId` und der Standard-Akku des neuen Modells gesetzt.

9. Akkuverwaltung

Es sind bis zu 16 Akku-Datensaetze vorgesehen. Ein Akku enthaelt:

- Name
- Zellzahl
- Kapazitaet
- Zyklen

Vier Akkus des alten Systems werden initial angelegt:

- 4S 5500 mAh
- 4S 2400 mAh
- 3S 2200 mAh
- 3S 1800 mAh

Im Akku-Menu kann ein Akku als aktuell eingesetzt markiert oder als Standard-Akku des aktiven Modells hinterlegt werden.

10. Menu-State-Machine

```

HOME
|
+-- Hauptmenue
    |
    +-- Monitor
    |   +-- Eingaenge
    |   +-- Runtime
    |   +-- Ausgaenge
    |
    +-- Trimmung
    +-- Expo / Rate
    +-- Mixer
    |   +-- Liste
    |   +-- Mixer bearbeiten
    |
    +-- Servos
    |   +-- Liste
    |   +-- Ausgang bearbeiten
    |
    +-- Regler
    |   +-- Poti-Liste
    |   +-- Poti-Zuordnung bearbeiten
    |
    +-- Modelle
    |   +-- Modellliste
    |   +-- Modellansicht
    |   +-- Aktivierung bestaetigen
    |
    +-- Akkus
    |   +-- Akkuliste
    |   +-- Akku bearbeiten / auswaehlen
    |

```

Vorgesehene Bedienung mit vorhandener Hardware

- Wipptaster 2 oben/unten: Cursor hoch/runter
- Wipptaster 1 links/rechts: Wert reduzieren/erhoehen oder Monitorseite wechseln
- rechter Taster kurz: Enter
- rechter Taster lang ab 700 ms: Zurueck

Diese Zuordnung kann spaeter ohne Aenderung der State-Machine durch einen anderen InputManager ersetzt werden.

11. Mixer

Jeder Mixer besitzt:

- Aktiv
- Quelle
- Ziel
- Basisgewicht -100..+100 Prozent
- Runtime-Zuschlag ueber ein zugewiesenes Poti
- Eingangstotzone 0..900
- Offset
- Modus `ADD` oder `CENTER\_SHIFT`

CENTER_SHIFT orientiert sich am Verhalten des alten Senders: Der zugemischte Wert verschiebt die Neutralposition und laesst moeglichst viel Restweg fuer die Hauptsteuerfunktion erhalten.

12. Display-Kommunikation Mega -> Nano

Die bereits optimierte Kommunikationsidee wird beibehalten und erweitert.

Paket:

```
FF FF
DisplayHeader
Payload
CRC8
80 80
```

Der Header enthaelt Payload-Typ, Payload-Laenge, Sequenznummer, UI-State und Summerstatus.

Der Nano:

- arbeitet ohne blockierende `readBytes()`-Aufrufe,
- verwirft unvollstaendige Pakete nach 100 ms,
- prueft Startmarker, Laenge, CRC8 und Endmarker,
- fordert maximal alle 50 ms ein neues Paket an,
- der Mega sendet zusaetzlich spaetestens alle 250 ms als Fallback,
- Monitorbalken werden nur differenziell aktualisiert.

Damit bleiben RC-Pfad und Display voneinander entkoppelt.

13. Wichtige Aenderungen gegenueber dem bisherigen Mega-Code

1. Trimmer sind keine CPPM-Kanaele 5 bis 8 mehr, sondern Runtime-Korrekturen der Steuerachsen.
2. CPPM-Kanaele 5 bis 8 stehen fuer echte Modellfunktionen zur Verfuegung.
3. `ppmISR[]` wird weiterhin atomar aktualisiert.
4. Alle CPPM-Werte werden auf 1000 bis 2000 us begrenzt.
5. A8 verwendet eine Dummy-ADC-Konvertierung nach dem A7/A8-MUX-Wechsel.
6. Zusatzpotis sind pro Modell frei zuweisbar.
7. Modell- und Akku-Konfigurationen liegen strukturiert und CRC-gesichert im EEPROM.
8. Laufende Poti-/Trimmerbewegungen erzeugen keine EEPROM-Schreibzyklen.
9. Menu und Display sind State-basiert statt an feste Diagnoseseiten gekoppelt.
10. Absolute Include-Pfade wurden entfernt.

14. Noch bewusst offene Punkte

Der vorliegende Stand ist ein Integrationsentwurf, kein freigegebener Flugbetriebsstand. Vor einem Einsatz am Modell sollten insbesondere noch abgeschlossen werden:

- exakte Kalibrierung aller Knueppel und Trimmer am realen Sender,
- Validierung der vier migrierten Modellprofile gegen die realen Servorichtungen,
- genaue Abbildung von Butterfly/Crow fuer Heron und Streamline,
- Fail-Safe-Verhalten bei Softwarefehler, Brownout und HF-Ausfall,
- sichere Sperre fuer Modellwechsel bei aktivem HF-Ausgang bzw. nicht minimalem Gas,
- ggf. Entprellung aller mechanischen Schalter,
- Test des EEPROM-Layouts auf dem realen Mega 2560,
- Kompilierung mit den konkret installierten Displaybibliotheken.

15. Testreihenfolge

1. Ohne HF-Modul starten und alle ADC-Werte im Monitor pruefen.
2. Trimmer bewegen: nur der Runtime-Wert darf sich aendern, EEPROM bleibt unberuehrt.
3. Zusatzpotis nacheinander verschiedenen Funktionen zuweisen.
4. CPPM-Ausgaenge mit Oszilloskop/Logic Analyzer pruefen.
5. Servorichtungen und Endpunkte pro Modell mit abgesteckten Ruderhoernern kontrollieren.
6. Modell wechseln, ausschalten, einschalten und pruefen, ob das zuletzt aktive Modell geladen wird.
7. Akku wechseln und Standard-Akku pro Modell pruefen.
8. Serielle Verbindung zum Nano unter Reset/Steckerunterbrechung testen; Display muss sich selbst wieder synchronisieren.